Graph-Structured Reinforcement Learning for Fair and Efficient CPU Process Scheduling

V. Saadhanaa
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
saadhanaa.2025@vitstudent.ac.in

Disha Vishal Chaudhari
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
disha.vishal2025@vitstudent.ac.in

Sankari M.
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sankari.m@vit.ac.in

Abstract—Conventional operating system schedulers such as First-Come First-Served (FCFS), Round Robin, and the Completely Fair Scheduler (CFS) achieve reasonable CPU utilisation but lack explicit mechanisms to prevent process starvation. This paper presents a reinforcement learning (RL) framework, GraphSAGE + MaskablePPO, that jointly optimises scheduling efficiency and fairness. At every scheduling tick, the system constructs a dynamic process-relationship graph encoding CPU contention, priority-class membership, and burst-history similarity among processes. A GraphSAGE [1] network aggregates this graph into a latent representation, which is then consumed by a Maskable Proximal Policy Optimisation (MaskablePPO) [2] agent. A soft-priority aging reward penalises the agent with increasing severity as any process's waiting time exceeds a configurable threshold, enabling the policy to learn anti-starvation behaviour without hand-crafted rules.

Evaluation across 500 seeded episodes with bootstrap confidence intervals demonstrates that the proposed method reduces average waiting time by 23.7 % and starvation rate by 24.4 percentage points relative to CFS [3] on uniform workloads. On exponential-burst (realistic) workloads, throughput remains comparable to CFS while starvation is reduced by 11.5 pp. A paired ablation study reveals that on realistic workloads, a flat-MLP agent without graph structure fails to significantly outperform CFS; the relational graph representation is essential for extracting scheduling-relevant signal.

Index Terms—CPU scheduling, reinforcement learning, graph neural networks, GraphSAGE, MaskablePPO, process scheduling, starvation, fairness, priority aging

I. Introduction

CPU process scheduling remains a fundamental challenge in operating system design. The Shortest Remaining Time First (SRTF) policy is provably optimal for minimising average waiting time, yet it requires a priori knowledge of future burst lengths—information unavailable to any practical scheduler. Consequently, production systems rely on heuristic policies. The Linux Completely Fair Scheduler (CFS) [3] is among the most effective: it maintains a per-process virtual runtime and dispatches the process with the smallest accumulated runtime, weighted by priority. While CFS achieves strong throughput, it provides no explicit guarantee against starvation. Although priority aging is discussed in standard textbooks [4], [5], CFS does not implement it.

Recent advances in deep reinforcement learning have demonstrated promising results for resource scheduling at the cluster and job levels—DeepRM [6] for cluster resource allocation, Decima [7] for Spark job DAGs, and RLScheduler [8] for HPC batch queues. However, these systems operate at a considerably coarser granularity than per-timeslice CPU dispatch. To the best of our knowledge, no prior work has combined a graph neural network with action masking and a fairness-aware reward for per-process OS-level scheduling.

This paper addresses that gap with three contributions:

1) A dynamic process graph reconstructed at every scheduling tick from observable process features, encoding contention, type similarity, and burst-history similarity relationships.
2) A GraphSAGE-based feature extractor integrated with MaskablePPO. GraphSAGE [1] is specifically suited to this setting, as it was designed for graphs whose topology changes between forward passes—processes arrive, block on I/O, resume, and terminate during execution.
3) A soft-priority aging reward that formalises the textbook concept of priority aging as an RL training signal, enabling the policy to learn anti-starvation behaviour from reward shaping alone.

II. Related Work

DeepRM [6] initiated the deep-RL-for-scheduling paradigm by training a REINFORCE policy over a two-dimensional image representation of cluster resources. RLScheduler [8] extended this approach with per-job feature vectors and PPO, matching HPC heuristics across six real workload traces. Neither system operates at the OS process level, nor does either incorporate graph structure or fairness objectives in the reward.

Decima [7] is the most closely related graph-based scheduler: it applies a custom GNN to static Spark DAGs and achieves a 21 % reduction in average job completion time. However, Decima's graph is a fixed dependency DAG that changes only upon job arrival; ours is rebuilt at every tick. Decima also lacks action masking and priority

mechanisms. L2D [9] applied GIN to manufacturing job-shop scheduling, and a recent survey [10] confirmed that GCN, GIN, and GAT dominate the scheduling literature—GraphSAGE has not previously appeared in any scheduling domain. Adimora et al. [11] applied Graph Attention Networks to 377K real supercomputer jobs, arguing that inductive methods are essential for variable-size queues; we share this motivation at a finer granularity.

On action masking, Huang and Ontáñón [12] demonstrated that setting invalid logits to $-\infty$ accelerates convergence by 57× compared to penalty-based approaches. Lassoued et al. [13] applied MaskablePPO to job-shop scheduling via Petri net masking—the closest prior work, although without a GNN, without a CPU scheduling domain, and with only 100 evaluation runs.

Regarding fairness, we are not aware of any RL scheduler that learns aging behaviour from reward shaping. A GPU cluster study [14] employs aging as a hand-coded heuristic; STUN [15] uses tabular Q-learning to globally tune CFS parameters. Neither constitutes a per-process dispatcher.

III. System Design

A. Environment

The environment simulates a single-CPU system with up to eight concurrent processes [16]. Each process alternates between CPU bursts and I/O waits. The scheduler observes all information available to a real OS dispatcher—waiting time, process type, CPU utilisation, recent burst history—but has no access to future burst lengths, preserving the causal constraint inherent in real scheduling.

Each process slot contributes 8 features to a flat 64-dimensional observation vector:

$$\mathbf{o}_i = \left[\, w_i,\ \tau_i,\ u_i,\ s_i,\ b_i^{\text{last}},\ \bar{b}_i,\ r_i,\ d_i \,\right]$$

representing waiting time, type, CPU used, schedule count, last and average burst length, ready flag, and blocked time, respectively. The action space is Discrete($N_{\max}+1$): the agent selects a process to dispatch or idles. Invalid actions (selecting a terminated or I/O-blocked process) are masked to $-\infty$ prior to the softmax.

B. Reward Function

At each tick, the agent receives a negative reward proportional to the total weighted waiting time of all ready processes not selected. When any process exceeds a starvation threshold W^*, an additional aging penalty is applied that grows linearly with the excess:

$$r_t = -\sum_{i \neq a_t,\ i\,\text{ready}} \Big[w_{\tau_i} + \alpha \cdot \max\!\big(0,\ W_i - W^*\big) \Big] \qquad (1)$$

where $\alpha = 0.5$ and $W^* = 10$. This formulation ensures that neglecting a long-waiting process incurs progressively greater cost, incentivising the policy to attend to starving processes without requiring explicit priority rules.

C. Dynamic Process Graph

At every tick, the graph is constructed from scratch. Processes are represented as nodes, connected by three families of undirected edges:

1) CPU Contention—connects processes that are both in the ready state, encoding the immediate scheduling conflict.
2) Type Similarity—connects processes sharing the same priority class ($\tau_i = \tau_j$), enabling the policy to coordinate scheduling within a process group.
3) Burst-History Similarity—connects processes whose average burst lengths are within one tick ($|\bar{b}_i - \bar{b}_j| \leq 1$). This edge family provides the policy with information about behaviourally similar processes without exposing future burst lengths.

All edges include self-loops. No state carries over between ticks.

D. GraphSAGE Feature Extractor

Two GraphSAGE layers perform neighbourhood aggregation via mean-pooling across two rounds of message passing:

$$\mathbf{h}_i^{(1)} = \text{ReLU}\big(W_1 \left[\mathbf{x}_i \,\|\, \overline{\mathbf{x}}_{N(i)}\right]\big) \qquad (2)$$

$$\mathbf{h}_i^{(2)} = \text{ReLU}\Big(W_2 \left[\mathbf{h}_i^{(1)} \,\|\, \overline{\mathbf{h}}_{N(i)}^{(1)}\right]\Big) \qquad (3)$$

where $W_1 \in \mathbb{R}^{32\times 20}$ and $W_2 \in \mathbb{R}^{16\times 64}$. The 8 node embeddings are concatenated and projected to a 48-dimensional vector, which feeds a standard PPO actor-critic network with two hidden layers of 64 units each. Fig. 1 illustrates the complete pipeline.

IV. Experimental Setup

Uniform workloads. CPU and I/O burst lengths are sampled from $\mathcal{U}\{1,5\}$, burst counts from $\mathcal{U}\{2,4\}$, and process types are assigned uniformly at random. Seeds 10 000–10 499 (500 episodes).

Realistic workloads. Exponential distributions with type-specific means derived from the Coffman–Kleinrock model [17] and standard OS textbook parameters [5]: SYSTEM (μ_{cpu}=2.5, μ_{io}=2.0), INTERACTIVE (μ_{cpu}=4.0, μ_{io}=5.0), BACKGROUND (μ_{cpu}=8.0, μ_{io}=2.0). Type distribution: 20/50/30 %. Seeds 20 000–20 499 (500 episodes).

Baselines. Six classical schedulers are evaluated: FCFS, Round Robin (q=2), Priority + Aging, MLFQ [4] (quanta 2/4/8), CFS [3], and Stride [18]. A flat-MLP MaskablePPO ablation (no graph) is included to isolate the contribution of the graph representation. SRTF with oracle access to future burst lengths serves as a theoretical upper bound.

Training. 150 000 timesteps, learning rate 3×10^{-4}, n_{steps}=2048, clip ratio ϵ=0.2. Multi-seed evaluation: 5 training seeds × 500 test workloads with bootstrap 95 % confidence intervals (5 000 resamples).

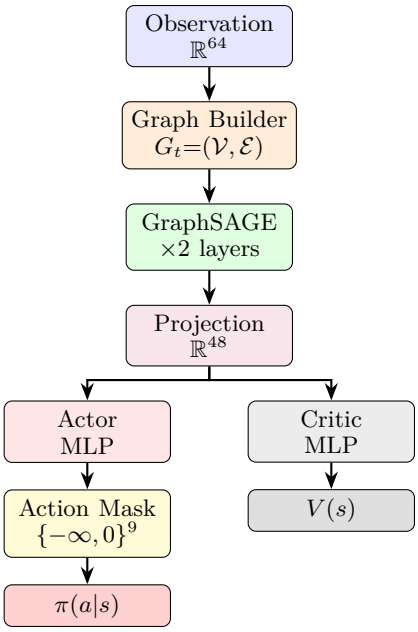


Fig. 1. End-to-end architecture of the proposed GraphSAGE + MaskablePPO framework. The process graph is reconstructed at every tick from observable features. Invalid actions are masked prior to the softmax.

TABLE I
Full benchmark across 500 seeded episodes. † denotes oracle access to hidden future burst lengths (theoretical bound, not a fair comparison).

Scheduler	Uniform		Realistic	
	Avg Wait	Starv.	Avg Wait	Starv.
FCFS	23.91	88.5 %	34.56	88.4 %
Round Robin	24.70	89.1 %	30.93	86.5 %
Priority + Aging	24.78	90.4 %	31.79	87.9 %
MLFQ	24.85	90.2 %	31.76	87.5 %
CFS	24.31	87.3 %	28.33	79.2 %
Stride	24.30	87.3 %	28.35	79.3 %
Plain MaskablePPO	19.30	65.4 %	28.28	68.5 %
GraphSAGE + MaskablePPO	18.54	62.9 %	28.00	67.7 %
SRTF†	14.49	53.1 %	17.32	52.4 %

TABLE II
Paired contribution decomposition. Δ denotes paired mean difference (negative indicates improvement). ✓ indicates the 95 % CI excludes zero.

WL	Transition	Δ Avg	95 % CI	Gap	Sig.
Unif.	Classical→RL flat	−4.6	[−5.0, −4.2]	49%	✓
	RL flat→RL+Graph	−0.8	[−1.0, −0.5]	8%	✓
	RL+Graph→SRTF	−4.1	[−4.3, −3.8]	43%	✓
Real.	Classical→RL flat	+0.6	[−0.3, +1.4]	n/a	×
	RL flat→RL+Graph	−0.7	[−1.4, −0.1]	7%	✓
	RL+Graph→SRTF	−10.9	[−11.9, −10.0]	99%	✓

V. Results

A. Full Benchmark

Table I presents the complete benchmark results. On uniform workloads, the proposed model achieves an average waiting time of 18.54 ticks (95 % CI [17.81, 19.25]) compared to 24.31 for CFS, representing a 23.7 % reduction. The starvation rate decreases from 87.3 % to 62.9 %, a reduction of 24.4 percentage points.

On realistic workloads, the average waiting time gap narrows to approximately 1.2 % (28.00 vs. 28.33), indicating comparable throughput. However, the starvation rate remains 11.5 percentage points lower (67.7 % vs. 79.2 %). This result suggests that while CFS is highly competitive on throughput with exponential burst distributions, the proposed method provides a meaningful fairness advantage.

B. Contribution Decomposition

The paired ablation study (Table II) isolates the contribution of each component. The same 500 workloads are evaluated under the flat-MLP agent and the full GraphSAGE agent, with paired differences computed via bootstrap resampling.

On uniform workloads, RL alone (without graph structure) closes approximately 49 % of the gap to the SRTF oracle. The addition of the graph contributes a further 8 %. Both improvements are statistically significant.

On realistic workloads, the flat-MLP agent performs slightly worse than CFS (paired difference +0.6, 95 % CI includes zero). However, incorporating the graph reverses this result to −0.7 (CI excludes zero). This finding indicates that on exponential-burst workloads, the graph representation is not merely beneficial—it is necessary for the RL agent to outperform CFS.

C. Multi-Seed Robustness

To assess reproducibility, five independently trained models are evaluated across 500 workloads each. The paired difference between the GraphSAGE and flat-MLP variants is −0.67 ticks (95 % CI [−1.12, −0.26]), confirming a modest but consistent advantage (Table III).

D. Visualisation

An interactive Streamlit dashboard was developed for qualitative inspection of scheduling behaviour. Fig. 3 displays the per-process state table and Gantt timeline; Fig. 4 visualises the dynamic process graph with all three edge families.

VI. Discussion

CFS proves to be a highly competitive baseline on realistic workloads. Its virtual-runtime weighting effectively handles heterogeneous exponential-burst processes, leaving minimal room for a flat-MLP policy to improve upon. The graph representation addresses this limitation by providing relational context. Through burst-history similarity edges, the policy identifies clusters of processes exhibiting similar behaviour, using this information as

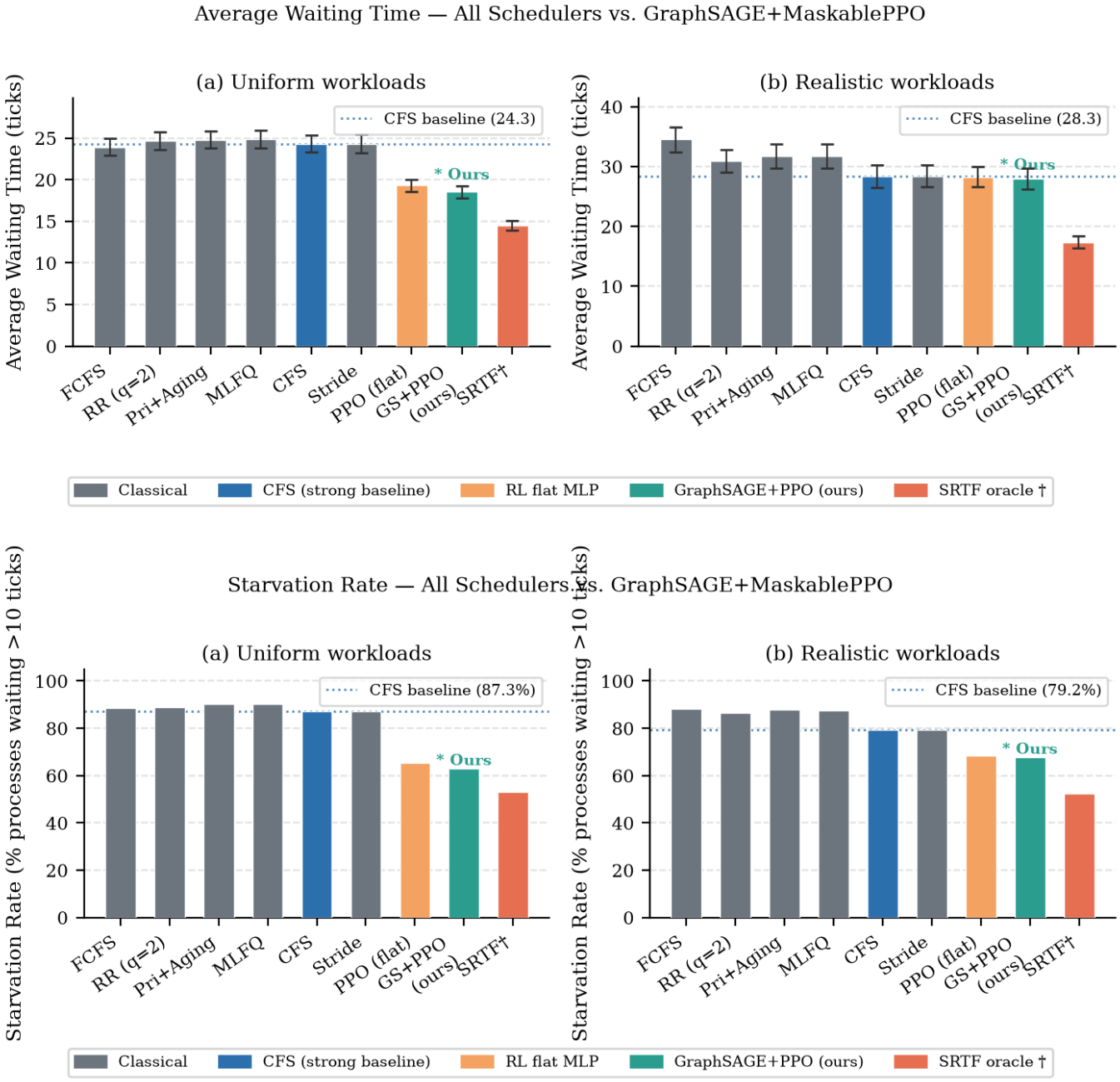


Fig. 2. Top: average waiting time across all schedulers (500 episodes, bootstrap 95 % CIs). Bottom: starvation rate. CFS baseline indicated by blue dotted line. The proposed model achieves the lowest non-oracle waiting time on uniform workloads and reduces starvation by 24.4 pp (uniform) and 11.5 pp (realistic) relative to CFS. SRTF utilises hidden burst lengths and serves as a theoretical lower bound.

TABLE III
Multi-seed evaluation (5 training seeds × 500 workloads).

Model	Avg Wait	95 % CI
Plain MaskablePPO	21.62	[21.20, 22.06]
GraphSAGE + MaskablePPO	20.96	[20.86, 21.07]
Paired diff. (G−P)	−0.67	[−1.12, −0.26]

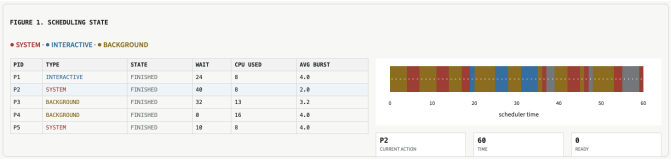


Fig. 3. Dashboard view of GraphSAGE + MaskablePPO on seed 42 with 5 processes. Left: per-process scheduling state. Right: Gantt timeline spanning 60 ticks. P2 (SYSTEM) accumulates the highest waiting time due to repeated I/O blocking; P4 (BACKGROUND) receives priority upon exceeding the aging threshold.

an approximate proxy for which processes will complete quickly. This constitutes a learned approximation of SRTF-style reasoning, derived entirely from observable history rather than hidden future information.

The remaining 10.91-tick gap to SRTF on realistic workloads is attributable primarily to an information-theoretic barrier. Exponential distributions exhibit a coefficient of variation of exactly 1.0; the next burst may differ from the historical average by several multiples. No causal policy—one that cannot observe future burst lengths—can recover this information. On uniform workloads, where variance is substantially lower, this gap narrows to 4.05 ticks.

The starvation reduction of 12–24 percentage points over CFS represents the most consistent finding across all experimental conditions. CFS contains no starvation threshold; the aging penalty (Eq. 1) provides the policy with a strong gradient to attend to long-waiting processes. To the best of our knowledge, this is the first RL-based scheduler that acquires anti-starvation behaviour purely through reward shaping.

A. Limitations

All experiments are conducted in a Python-based simulator; no kernel integration has been performed. The current implementation is limited to 8 concurrent processes due to the fixed observation size. The "realistic" workloads are generated from textbook distributions rather than captured from production /proc traces. Additionally, all three edge types employ the same mean-pooling aggregation; relation-specific aggregation kernels such as R-GCN [19] may yield further improvements.

VII. Conclusion

This paper presents GraphSAGE + MaskablePPO, a graph-structured reinforcement learning framework for CPU process scheduling that jointly optimises efficiency and fairness. Two principal findings emerge. First, the

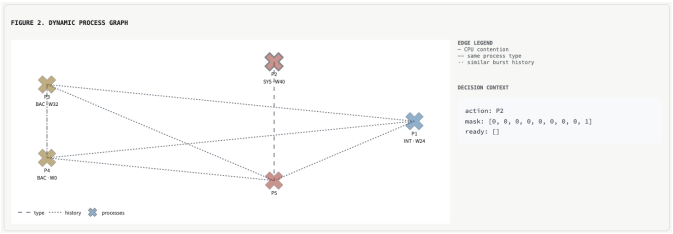


Fig. 4. Dynamic process graph as rendered in the dashboard. Three edge families are shown: CPU contention (solid), type similarity (dashed), and burst-history similarity (dotted). The graph is reconstructed at every tick and aggregated by GraphSAGE into a 48-dimensional policy input.

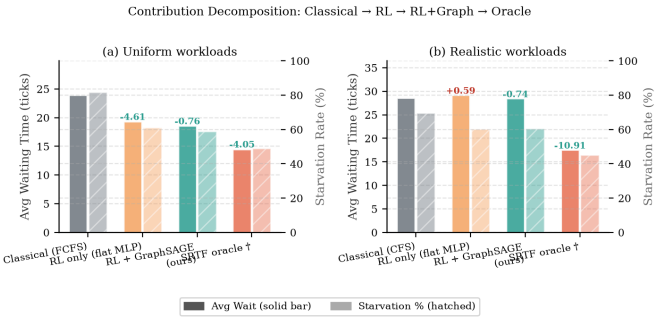


Fig. 5. Contribution decomposition (Classical → RL → RL+Graph → SRTF). Solid bars: average waiting time. Hatched bars: starvation rate. On realistic workloads, RL alone does not significantly outperform CFS (+0.6, n.s.), but the addition of the graph yields a significant improvement (−0.7).

soft-priority aging reward consistently reduces starvation by 12–24 percentage points relative to CFS and all classical baselines across every experimental condition. Second, the graph representation is essential: on realistic workloads—those most representative of production OS environments—a flat RL agent cannot significantly outperform CFS. The relational structure encoded by the dynamic process graph provides the signal necessary to identify scheduling opportunities that a flat observation vector misses.

Future work will focus on eBPF [20] integration for deployment within a real Linux kernel, extension to variable-length process queues via sparse graph attention mechanisms, and evaluation on production workload traces.

References

[1] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 1024–1034.

[2] Costa-juárez et al., "sb3-contrib: Contrib package for stable-baselines3 (MaskablePPO)," https://sb3-contrib.readthedocs.io, 2022.

[3] I. Molnar, "Completely fair scheduler," Linux Kernel Documentation, 2007. [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html

[4] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, 1st ed. Arpaci-Dusseau Books, 2023. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/
[5] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Wiley, 2018.
[6] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets). ACM, 2016.
[7] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in Proceedings of ACM SIGCOMM. ACM, 2019.
[8] D. Zhang, D. Dai, Y. He, F. S. Bao, and B. Xie, "RLScheduler: An automated HPC batch job scheduler using reinforcement learning," in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). IEEE, 2020.
[9] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to dispatch for job shop scheduling via deep reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1621–1632.
[10] I. Echeverria, M. Murua, and R. Santana, "A review of graph neural networks for job shop scheduling problems," arXiv preprint arXiv:2406.14096, 2024.
[11] E. N. Adimora et al., "Deep reinforcement learning for efficient HPC job scheduling," in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). IEEE, 2024.
[12] S. Huang and S. O. nón, "A closer look at invalid action masking in policy gradient algorithms," in Proceedings of the Florida Artificial Intelligence Research Society Conference (FLAIRS), vol. 35, 2022.
[13] Y. Lassoued et al., "Dynamic job shop scheduling with maskable PPO and Petri net masking," Expert Systems with Applications, vol. 263, 2026.
[14] Anonymous, "Aging-based fairness in GPU cluster scheduling," arXiv preprint arXiv:2512.10980, 2025.
[15] J. Lee, S. Jung, and H. Jo, "STUN: Reinforcement-learning-based optimization of kernel scheduler parameters," Applied Sciences, vol. 12, no. 14, p. 7072, 2022.
[16] M. Towers et al., "Gymnasium," https://gymnasium.farama.org, 2023.
[17] E. G. Coffman and L. Kleinrock, "Feedback queueing models for time-shared systems," Journal of the ACM, vol. 15, no. 4, pp. 549–576, 1968.
[18] C. A. Waldspurger and W. E. Weihl, "Stride scheduling: Deterministic proportional-share resource management," MIT, Tech. Rep. MIT/LCS/TM-528, 1995.
[19] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in Proceedings of the European Semantic Web Conference (ESWC). Springer, 2018.
[20] Linux Foundation, "eBPF – extended berkeley packet filter," https://ebpf.io, 2024.